Name: **SOLUTION** UID#: DirectoryID:

CMSC216: Practice Exam 2B SOLUTION

Fall 2026

University of Maryland

Exam period: 20 minutes

Points available: 40

Problem 1 (15 pts): Nearby is a C function `col_update()` with associated data and documentation. **Re-implement this function in x86-64 assembly** according to the documentation given. Follow the same flow provided in the C implementation. The comments below the `colinfo_t` struct give information about how it lays out in memory and as a packed argument.

Indicate which registers correspond to which C variables.

```
### SOLUTION:
.text
.global col_update

# YOUR CODE BELOW
col_update:
### rdi is info ptr; esi is niters
        movl    0(%rdi),%r8d    # cur = info->cur
        movl    4(%rdi),%edx    # step= info->step
        cmpl    $0,%r8d         # if(cur <= 0)
        jg      .INITLOOP       # false: not an error
        movl    $-1,%eax        # error case
        ret                     # return -1
.INITLOOP:
        movl    $0, %ecx        # i = 0
.LOOPTOP:
        cmpl    %esi, %ecx      # for(..; i<niters; ..)
        jge     .RETURN         # done with loop
        addl    $1,%edx         # step++
        testl   $0x01,%r8d      # if(cur%2 == 1)
        jz      .EVEN           # go to even case
## ODD CASE (fall through)
        imull   $3,%r8d         # odd: cur *= 3
        addl    $1,%r8d         # odd: cur += 1
        jmp     .INCREMENT        # jump over even
.EVEN:
        sarl    $1,%r8d         # even: cur /= 2
.INCREMENT:
        incl    %ecx            # i++
        jmp     .LOOPTOP
.RETURN:
        movl    %r8d,0(%rdi)    # info->cur = cur;
        movl    %edx,4(%rdi)    # info->step= step;
        movl    $0,%eax         # success
        ret                     # return 0
```

```
typedef struct{
  int cur;
  int step;
} colinfo_t;
// |       | Byte |   Byte | Packed |
// | Field | Size | Offset |   Bits |
// |-------+------+--------+--------|
// | cur   |    4 |     +0 |   0-31 |
// | step  |    4 |     +4 |  32-63 |

int col_update(colinfo_t *info, int niters){
  // Performs ‘niters‘ updates the cur/step fields
  // in struct pointed at by ‘info‘. If
  // infor->cur is invalid, makes no changes
  // and returns -1 to indicate an
  // error. Otherwise performs successive ‘niters‘
  // odd or even updates on ecur and increments
  // step each time; then returns 0 for success.

  int cur  = info->cur;
  int step = info->step;
  if(cur <= 0){
    return -1;
  }
  for(int i=0; i<niters; i++){
    step++;
    if(cur % 2 == 1){
      cur = cur*3+1;
    }
    else{
      cur = cur / 2;
    }
  }
  info->cur  = cur;
  info->step = step;
  return 0;
}
```

Problem 2 (15 pts): Below is an initial register/memory configuration along with snippets of assembly code. Each snippet is followed by a blank register/memory configuration which should be filled in with the values to reflect changes made by the preceding assembly. The code is continuous so that POS A is followed by POS B.

```
SOLUTION:            addl  %edi, %esi
                     subq  $8, %rsp          movq  $1, %rdi
                     movl  $100, 4(%rsp)     addl  %esi, (%rsp,%rdi,4)
                     movl  $300, 0(%rsp)     leaq  8(%rsp), %rdi
                     addl  (%rsp), %eax      addl  (%rdi), %eax
INITIAL              # POS A                 # POS B
|-------+-------|    |-------+-------|       |-------+-------|
| REG   | Value |    | REG   | Value |       | REG   | Value |
|-------+-------|    |-------+-------|       |-------+-------|
| rax   |    10 |    | rax   |   310 |       | rax   |   560 |
| rdi   |    20 |    | rdi   |    20 |       | rdi   | #3032 |
| rsi   |    30 |    | rsi   |    50 |       | rsi   |    50 |
| rsp   | #3032 |    | rsp   | #3024 |       | rsp   | #3024 |
|-------+-------|    |-------+-------|       |-------+-------|
| MEM   | Value |    | MEM   | Value |       | MEM   | Value |
|-------+-------|    |-------+-------|       |-------+-------|
| #3032 |   250 |    | #3032 |   250 |       | #3032 |   250 |
| #3028 |     1 |    | #3028 |   100 |       | #3028 |   150 |
| #3024 |     2 |    | #3024 |   300 |       | #3024 |   300 |
| #3020 |     3 |    | #3020 |     3 |       | #3020 |     3 |
|-------+-------|    |-------+-------|       |-------+-------|
```

Problem 3 (10 pts): Rover Witer is writing an assembly function called `compval` which he will use in C programs. He writes a short C `main()` function to test `compval` but is shocked by the results which seem to defy the C and assembly code. Valgrind provides no insight for him. **Identify why** Rover's code is behaving so strangely and fix `compval` so it behaves correctly.

Sample Compile / Run:

```
> gcc ...                | REGS  | VALS  |
> a.out                  | rdi   | #2032 |-+
expect: 0       |--------+-------+-------| |
actual: 19      |        | STACK |       | |
expect: 0       | expect | #2036 | 19->0 | |
actual: 50      | actual | #2032 | 19    |<+
```

SOLUTION: The `movq` *instruction at line 7 of* `compval` *writes 8 bytes. This is inappropriate as a 4-byte int is supposed to be written. Apparently the stack layout in* `main()` *has the variable* `actual` *at a memory address immediately below variable* `expect` *so that on writing 8 bytes, the low order 4 bytes correctly get written to* `actual` *but the high order 4 bytes (all 0's for small values) overwrite the variable* `expect` *leaving it as 0. The fix for this is to use* `movl %eax, (%rdx)` *which will write 4 bytes, filling only* `actual`.

```
1  // compval_main.c
2  #include <stdio.h>
3
4  void compval(int x, int y, int *val);
5  // compute something based on x and y
6  // store result at int pointed to by val
7
8  int main(){
9    int expect, actual;
10
11   expect = 7 * 2 + 5;     // expected value
12   compval(7, 2, &actual); // actual result
13   printf("expect: %d\n",expect);
14   printf("actual: %d\n",actual);
15
16   expect = 5 * 9 + 5;     // expected value
17   compval(5, 9, &actual); // actual result
18   printf("expect: %d\n",expect);
19   printf("actual: %d\n",actual);
20
21   return 0;
22 }
```

```
1  # compval_asm_corrected.s
2  .text
3  .global compval             # imul/add should be 32-bit
4  compval:                    # biggest problem is movq
5          imulq   %rdi,%rsi   # likely: imull %edi, %esi
6          addq    $5,%rsi     # likely: addl $5, %esi
7          movl    %esi,(%rdx) # was movq %rsi, (%rdx)
8          ret                 # MUST do a movl, not movq
```